

De inwendige werking van een CPU

Inleiding

Het doel van deze module is het begrijpen van de interne werking van een zeer eenvoudige CPU. Hiervoor zal je de hardware goed moeten bestuderen, zodat je begrijpt wat er gebeurt tijdens het uitvoeren van een programma in machinetaal. Het lijkt misschien zeer specialistisch en verregaand detailwerk, maar door af te dalen tot op bit- en poortniveau is het de bedoeling dat je de werking van deze zeer eenvoudige processor kan extraheren naar zeer complexe CPU's zoals een moderne multicore Intel processor. Elke programmeertaal, hoe ingewikkeld ook, wordt ALTIJD omgevormd tot bits en bytes: machinetaal. Moderne compilers leveren meestal goed werk, maar in sommige gevallen kan het nog steeds nodig zijn om (stukjes) superperformante code te schrijven op "laag niveau", meestal voor zeer specifieke hardware zoals embedded systemen.

Als educatief voorbeeld is er een fictieve CPU beschreven in VHDL (SimpleCPU) en het is de bedoeling dat je deze VHDL in simulatie bestudeert. Je zal dus geen FPGA moeten configureren voor deze module en dus ook niet moeten synthetiseren. Je mag dit uiteraard altijd doen, maar vermits je dan de inwendige signalen niet meer kan bestuderen, kan je dit evengoed doen met een bestaande CPU. Dit doe je dan ook in de andere twee modules, waar de focus meer ligt op de assembler code en de communicatie met de randhardware (periferie).

De SimpleCPU

Inleiding

Het is belangrijk om te weten dat de CPU die hier in VHDL beschreven wordt een fictieve processor is die enkel gemaakt werd voor educatieve doeleinden. Het is een CPU die slechts kan werken met een kleine twintigtal instructies van 16-bits. Als opgave zal je trouwens een extra instructie hieraan moeten toevoegen. Subroutines, stack, heap, interrupts worden allemaal niet ondersteund, wat bij echte CPU's wel altijd het geval is. Alle CPU's hebben echter wel een gelijkaardige opbouw aan de hand van een ALU, decoder, program counter, instruction register, enz.

Nog een belangrijk verschil met reële CPU's is de gebruikte instructiecyclus in dit ontwerp. Hier wordt namelijk gewerkt met vier toestanden: Fetch, Decode, Execute en Increment. Tijdens deze laatste toestand wordt de inwendige ALU van de processor gebruikt om de program counter te verhogen, terwijl dit in normale CPU's tijdens de Decode of Execute toestand gebeurt aan de hand van een aparte opteller. Daarnaast gebruiken de meeste CPU's ook pipelining, waarbij meerdere instructiecycli in parallel met elkaar uitgevoerd worden. Voor de eenvoud gebruikt de SimpleCPU dit allemaal niet.

Verder staat het programmeergeheugen ook mee in de CPU, wat zeker niet gebruikelijk is. In een computer staan de instructies in het werkgeheugen (de uitbreidbare groene latjes) en worden soms gekopieerd naar het interne cache geheugen. Hierover zal je meer in de theorie te weten komen, maar voor de eenvoud wordt hier geen gebruik van gemaakt. De algoritmes die gebruikt worden in cache controllers zijn trouwens meestal proprietary en zeker niet eenvoudig.

In de twee andere modules van dit vak (assembler en periferie) zal er echter gebruik gemaakt worden van microcontrollers, waarbij het dan weer wel gebruikelijk is dat het programmeergeheugen mee in de controller verwerkt zit.

De ALU is ook redelijk beperkt qua aritmetische bewerkingen, maar daarover verder meer.

Instructies

De volgende tabel bevat alle instructies die ondersteund worden door de SimpleCPU. De bijhorende machinetaal wordt opgesplitst in een 8-bit opcode en 8-bit operand. Hierbij staat “kk” steeds voor een constante, “mm” voor een geheugenadres en “aa” voor een instructie adres. “XX” staat zoals steeds voor “don’t care”, maar in de gegeven assembler code worden hier altijd nullen voor gebruikt.

Omwille van het educatieve karakter van deze fictieve processor krijgen de instructies hier veelzeggende namen, terwijl dit in bestaande processoren zeer dikwijls afkortingen van één woord zijn. Zo zal bijvoorbeeld de “READ MEM” instructie dikwijls vervangen worden door een “MOV” instructie in échte assembler code. Het loont zeker de moeite om eens te zoeken naar de overeenkomstige assembler instructies van een echte processor (zoals de 8-bit AVR microcontroller van Atmel uit de andere module van dit vak).

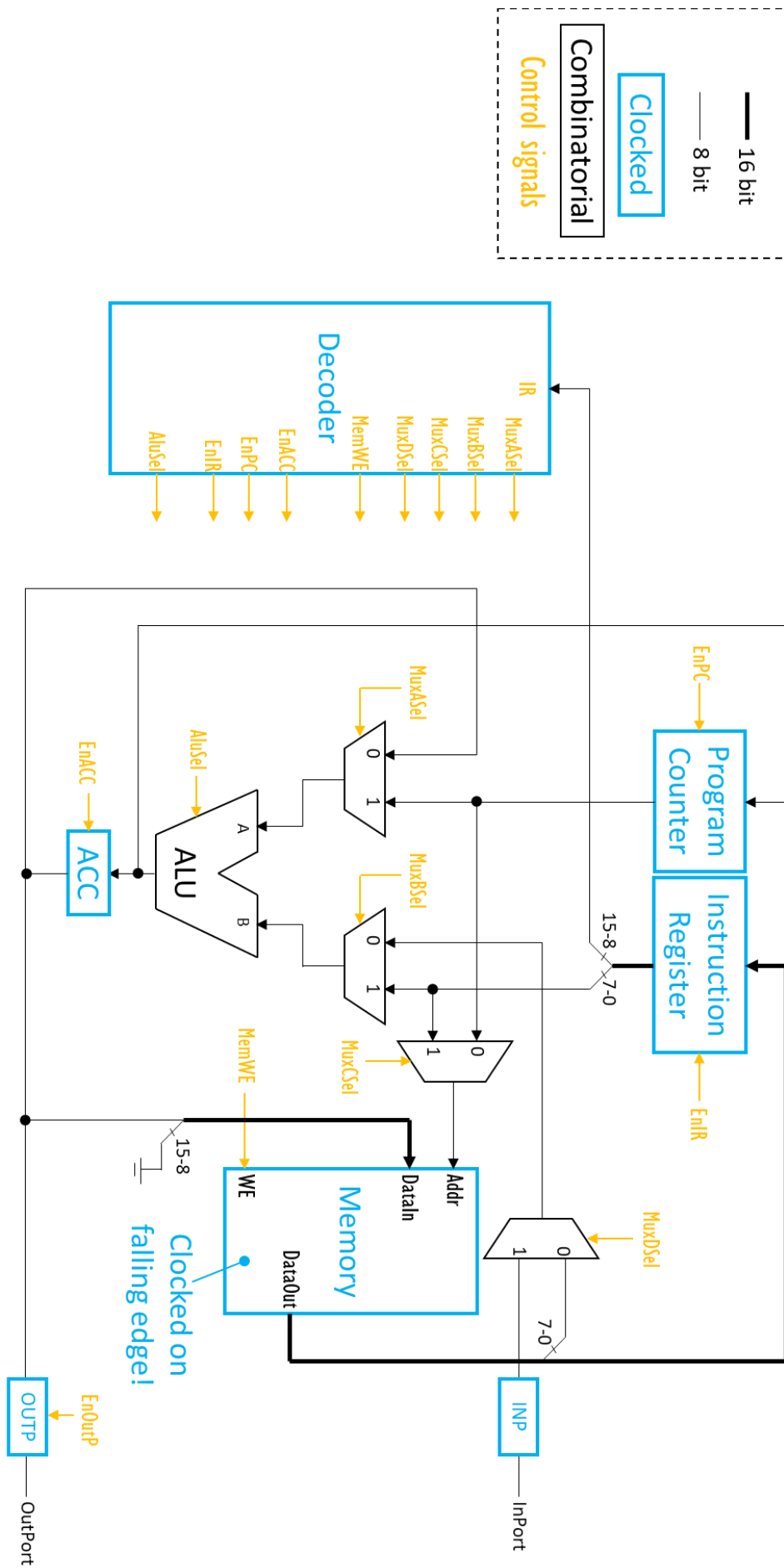
Instructie	Opcode	Operand	Functie
LOAD ACC kk	0000 XXXX	KKKK KKKK	Bewaar de constante K in de accumulator
AND ACC kk	0001 XXXX	KKKK KKKK	Voer een bitgewijze EN uit met K op de accumulator
ADD ACC kk	0010 0XXX	KKKK KKKK	Tel de waarde K op bij de accumulator
ADD IND mm	0010 1XXX	MMMM MMMM	Tel de waarde op geheugenadres M op bij de accumulator
SUB ACC kk	0011 XXXX	KKKK KKKK	Trek de waarde K af van de accumulator
READ MEM mm	0100 XXXX	MMMM MMMM	Stop de waarde van adres M in de accumulator
WRITE MEM mm	0101 XXXX	MMMM MMMM	Schrijf de waarde van de accumulator naar adres M
INPUT ACC	0110 XXXX	XXXX XXXX	Stop de waarde van de input port in de accumulator
OUTPUT ACC	0111 XXXX	XXXX XXXX	Schrijf de waarde van de accumulator naar de output port
JUMP U aa	1000 XXXX	AAAA AAAA	Spring onvoorwaardelijk naar instructie A
JUMP Z aa	1001 00XX	AAAA AAAA	Spring naar instructie A als de Z-vlag ‘1’ is
JUMP C aa	1001 10XX	AAAA AAAA	Spring naar instructie A als de C-vlag ‘1’ is
JUMP NZ aa	1001 01XX	AAAA AAAA	Spring naar instructie A als de Z-vlag ‘0’ is
JUMP NC aa	1001 11XX	AAAA AAAA	Spring naar instructie A als de C-vlag ‘0’ is
JUMP IND aa	1011 XXXX	MMMM MMMM	Spring naar de instructie waarvan het adres op M staat

Verderop wordt er uitgelegd hoe deze instructies exact geïmplementeerd worden. Je zal zien dat de bijhorende VHDL helemaal niet zo moeilijk is. De hardware voor deze processor is misschien verrassend eenvoudig, maar het gaat hier wel om een zeer vereenvoudigde versie van een CPU.

Je ziet dat er qua codering nog veel ruimte is voor extra instructies (mogelijke bitcombinaties in de opcode kolom) en je zal er in de opgaven ook effectief eentje moeten toevoegen.

Opbouw

Op de volgende pagina kan je het blokschema van de SimpleCPU vinden. De belangrijkste blokken worden daarna besproken.



De dikke lijnen duiden een 16-bit signaal aan, terwijl de dunnen lijnen slechts 8 bits bevatten. De blauwe blokken werken synchroon met de klok (flip flops) en de rest is combinatorische logica. Controlesignalen staan in het oranje aangeduid. De gebruikte blokken zijn:

- Memory: het geheugen waarin zowel de instructies als gegevens van het programma staan. Dit blok werkt op de dalende flank van de klok, zodat de opgevraagde data op tijd (één klokflank vroeger) ter beschikking staat voor de rest van de geklokte blokken.
- Program Counter: een register waarin de huidige program counter bewaard wordt. Tijdens de increment toestand wordt deze verhoogd aan de hand van de ALU, wat zeker niet gebruikelijk is in standaard CPU's.
- Instruction Register: dit 16-bit register bevat steeds de huidige instructie, opgesplitst in twee delen:
 - Bits 15 t.e.m. 8: de opcode van de instructie. Hieruit wordt in het Decoder blok afgeleid welke operatie er exact uitgevoerd moet worden.
 - Bits 7 t.e.m. 0: de operand van de instructie. Hierin staat het adres of data waarmee de instructie juist moet werken.
- ALU: de Arithmetic Logic Unit in dit ontwerp kan volgende bewerkingen uitvoeren, waarbij A en B de respectievelijke ingangen zijn:
 - A: in geval van de Write Mem instructie wordt enkel A bekeken door de ALU.
 - B: in geval van een Input, Load of Jump-instructie wordt de geheugendata doorgelaten via B.
 - A + B: de positief binaire optelling tussen A en B.
 - A - B: de Carry Flag betekent in dit geval niet altijd een overflow (zie verder).
 - A & B: een bitgewijze EN-functie tussen A en B.
 - A + 1: wordt gebruikt om de Program Counter te verhogen.

Deze ALU beschikt over twee vlaggen, namelijk:

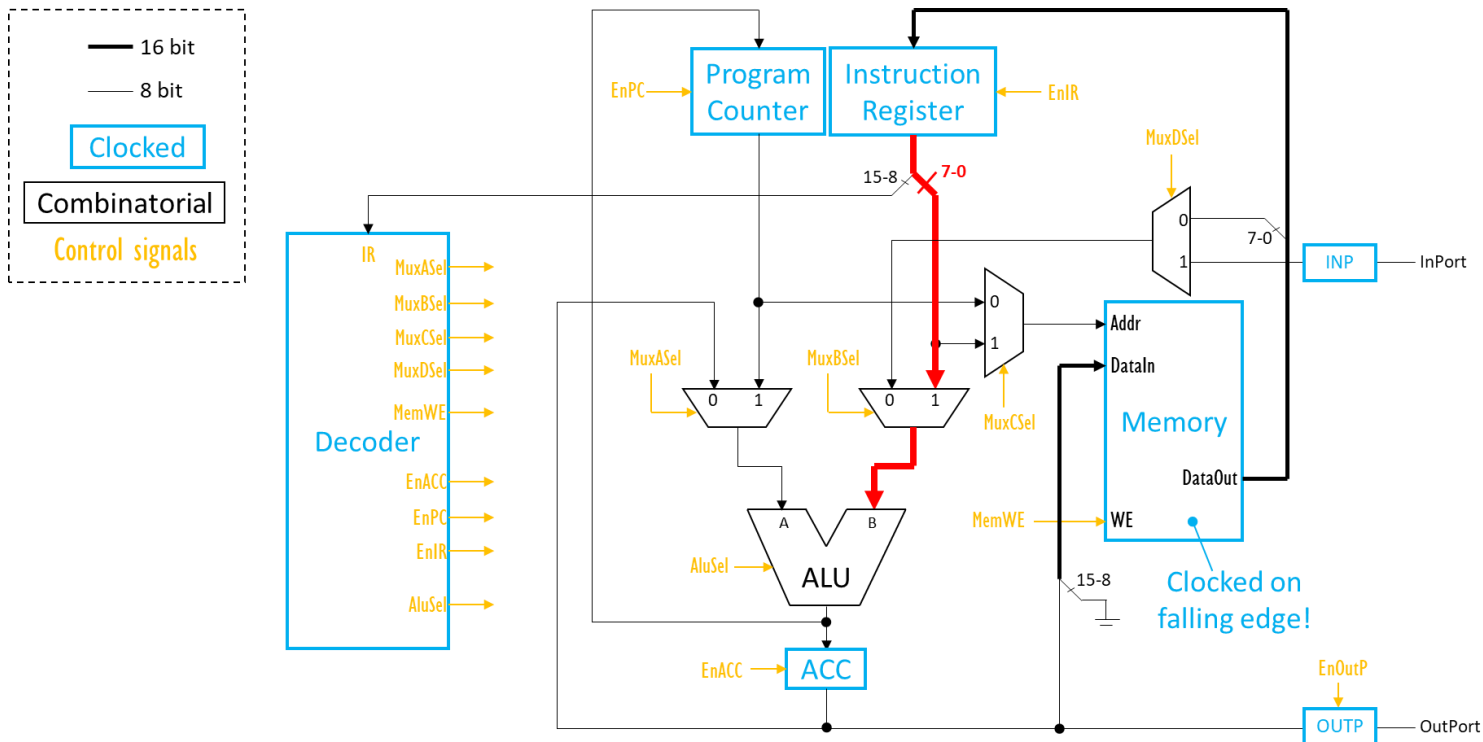
- CF: de Carry Flag: bij een Add of Sub instructie wordt hier de waarde van de hoogste (meest linkse) carry bit doorgegeven. Deze kan dan gebruikt worden om voorwaardelijke Jump-instructies uit te voeren (zo zal bijvoorbeeld JUMPNC enkel de Jump uitvoeren als de CF gelijk is aan 0).
- ZF: de Zero Flag: na iedere instructie wordt er gekeken of de uitgang van de ALU gelijk is aan 0. Indien dit het geval is, dan wordt deze vlag op 1 gezet. Ook dit kan dan gebruikt worden om voorwaardelijke Jump-instructies uit te voeren (zo zal bijvoorbeeld JUMPZ enkel de Jump uitvoeren als de ZF gelijk is aan 1).

Merk op dat bij een Sub-instructie de bewerking $A + (-B)$ zal worden berekend, door de two's complement waarde van B op te tellen met A. Zoals je weet van het vak Basis Digitale Elektronica 1, betekent een 1 op CF in dat geval niet altijd een overflow. Je zal dus zelf nog de juiste bewerkingen moeten doen om te bepalen of er een overflow plaatsvond.

- Decoder: dit blok bekijkt de opcode van de binnenkomende instructie en bepaalt wat er exact moet gebeuren aan de hand van de controlesignalen, die naar alle betreffende blokken gaan. Onderstaande tabel geeft aan wat de waarden van deze controlesignalen juist zijn bij iedere ondersteunde instructie.
 - De kolom "Cyclus" bevat de afkortingen F, D, E, I die naar de respectievelijke toestanden (Fetch, Decode, Execute en Increment) van de instructiecyclus refereren.
 - De kolom "CF" en "ZF" bevatten de waarden van de Carry Flag en de Zero Flag.

Instructie	Cyclus	CF	ZF	MuxA	MuxB	MuxC	MuxD	MemWE	EnACC	EnPC	EnIR	EnOutp	ALU
LOAD ACC	D	X	X	0	1	0	0	0	0	0	0	0	B
	E	X	X	0	1	0	0	0	1	0	0	0	B
ADD ACC	D	X	X	0	1	0	0	0	0	0	0	0	A + B
	E	X	X	0	1	0	0	0	1	0	0	0	A + B
ADD IND	D	X	X	0	0	1	0	0	0	0	0	0	A + B
	E	X	X	0	0	1	0	0	1	0	0	0	A + B
AND ACC	D	X	X	0	1	0	0	0	0	0	0	0	A & B
	E	X	X	0	1	0	0	0	1	0	0	0	A & B
SUB ACC	D	X	X	0	1	0	0	0	0	0	0	0	A - B
	E	X	X	0	1	0	0	0	1	0	0	0	A - B
READ MEM	D	X	X	0	0	1	0	0	0	0	0	0	B
	E	X	X	0	0	1	0	0	1	0	0	0	B
WRITE MEM	D	X	X	0	0	1	0	0	0	0	0	0	A
	E	X	X	0	0	1	0	1	0	0	0	0	A
INPUT	D	X	X	0	0	0	1	0	0	0	0	0	B
	E	X	X	0	0	0	1	0	1	0	0	0	B
OUTPUT	D	X	X	0	0	0	0	0	0	0	0	0	B
	E	X	X	0	0	0	0	0	0	0	0	1	B
Jump U	E	X	X	0	0	0	0	0	0	1	0	0	B
Jump Z	E	X	1	0	0	0	0	0	0	1	0	0	B
Jump NZ	E	X	0	0	0	0	0	0	0	1	0	0	B
Jump C	E	1	X	0	0	0	0	0	0	1	0	0	B
Jump NC	E	0	X	0	0	0	0	0	0	1	0	0	B
Jump IND	D	X	X	0	0	1	0	0	0	0	0	0	B
	E	X	X	0	0	1	0	0	0	1	0	0	B
X	I	X	X	1	0	0	0	0	0	<i>Jump</i>	0	0	A+1
X	F	X	X	0	0	0	0	0	0	0	1	0	0

Om deze tabel te interpreteren, zullen we bijvoorbeeld de Load instructie bekijken. Als je de verschillende controlesignalen invult in het blokschema, dan krijg je de rood gemarkeerde datapaden in onderstaand schema.



Zoals je ziet, zal bij een LOAD het operand-gedeelte (de laagste 8 bits) van de instructie geladen worden. Tijdens de Decode cyclus wordt deze data klaargezet aan de accumulator, zodat daarna in de Execute cyclus **EnACC** actief wordt en de data in de accumulator geklokt wordt.

Op deze manier kan je dit nagaan voor iedere instructie in het SimpleCPU ontwerp.

Het timer blok

Zoals dit ook in andere CPU's het geval is, zit er niet altijd effectief geheugen op iedere plaats van de geheugenmap. Zo zit er ook in de SimpleCPU een Timer blok op de geheugenplaatsen 0xFD, 0xFE en 0xFF. Een read of write naar deze adressen resulteert dus in communicatie met dit Timer blok in plaats van met het geheugen.

Bestudeer zeker de VHDL code van dit blok, zodat je onderstaande werking begrijpt:

Timer control register (0xFD)

Bit	Default (binair)	R/W	Functie
7-5	001	R/W	Stapgrootte van de timer.
4	0	R/W	0: de timer is uitgeschakeld (behoudt zijn waarde). 1: de timer is ingeschakeld (telt onmiddellijk verder).
3	0	R/W	0: bij overflow zal de timer opnieuw starten bij 0. 1: bij overflow blijft de timer op zijn laatste waarde staan.
2	0	R	0: de timer heeft zijn maximum waarde nog niet bereikt. 1: de timer heeft zijn maximum waarde bereikt.
1	0	W	Een '1' schrijven naar deze bit resulteert in het wissen van bit 2. Deze bit lezen geeft altijd een '0'.
0	0	W	Een '1' schrijven naar deze bit resulteert in het resetten van de timer. Deze bit lezen geeft altijd een '0'.

Timer MSB register (0xFE)

Bit	R/W	Functie
7-0	R	De most significant byte van de huidige timer waarde.

Timer LSB register (0xFF)

Bit	R/W	Functie
7-0	R	De least significant byte van de huidige timer waarde.

Het timer blok werd zodanig ontworpen dat het enkele veel voorkomende functionaliteiten bevat wat betreft de registers waarmee je kan communiceren. Zo kan je zien dat niet iedere bit gelezen en/of geschreven kan worden (R/W: lezen en schrijven; R: enkel lezen; W: enkel schrijven). Bit 2 van het timer control register wordt '1' en blijft '1' tot het terug gewist wordt door een '1' naar bit 1 te schrijven. Experimenteer zeker genoeg met deze bits om de werking ervan te begrijpen.

Je zal hierbij merken dat je in de problemen komt als je niet alle bits van het register wil overschrijven, bijvoorbeeld wanneer je enkel de timer wil resetten (een '1' schrijven naar bit 0) zonder de waarden van de andere bits te overschrijven. Hierover meer in onderstaande opdrachten.

De timer heeft 16 bits, hoewel de CPU slechts met een 8-bit geheugenmap werkt. Hierdoor wordt de huidige waarde van de timer opgesplitst in 2 plaatsen van de geheugenmap. Dit zorgt dan weer voor problemen met accurate uitlezing wanneer de timer nog aan het lopen is. De timerwaarde zal veranderd zijn in de tijd tussen het lezen van de MSB en de LSB. Dit zijn allemaal problemen waar je bij "hoog niveau" software niet mee te maken krijgt en die op assembler niveau opgelost moeten worden.

Merk op dat bij realistische timer registers gewerkt wordt met interrupts, zodat er niet steeds manueel gekeken moet worden naar de huidige waarde van het register. Hierover wordt verder uitgeweid in de module omtrent de periferie.

Opgaven

Inleiding

De opgaven van deze module dienen om je de werking van deze CPU te leren en je moet ze **niet** afgeven. Tijdens de laatste zitting van het semester zal je enkele individuele oefeningen moeten maken die gelijkaardig zijn aan de onderstaande. Hierop zal je gequoteerd worden, dus het is in je eigen belang dat je ze goed probeert te maken én te begrijpen tijdens de eerste twee zittingen. Als je veel “samenwerkt”, dan is de kans groot dat je de individuele oefeningen niet zal kunnen oplossen. Als je alle oefeningen zelf maakt en veel vragen stelt over de dingen die je niet begrijpt, dan zou je de individuele oefeningen zonder problemen moeten kunnen oplossen. **ALLES uit onderstaande oefeningen zal je nodig hebben op de test!**

Tip: tijdens deze individuele oefening mag je alles gebruiken behalve communicatiemiddelen, dus maak misschien enkele kleine verslagen voor jezelf van onderstaande opgaven.

● Opgave 1

Om te beginnen, simuleer je (in Vivado) de SimpleCPU en bekijk je zijn inwendige signalen: de controlesignalen, maar ook de datapaden en adrespaden doorheen het ontwerp. Zorg er ook voor dat je de gegeven assembler code (van opgave 1) begrijpt en dat je dus weet wat het programma eigenlijk juist doet. Je kan hiervoor de gegeven testbench gebruiken¹. Op Blackboard kan je ook een zogenaamde “simulation waveform configuration file” vinden die je kan openen vanuit je simulatie. Hierin staan reeds interessante (inwendige) signalen gegroepeerd om je te helpen een overzicht te verkrijgen. Eens je simulatie open staat, klik je hiervoor op “File => Simulation Waveform => Open configuration” en kies je vervolgens het betreffende .wcfg bestand.

In Vivado staat er echter een vervelende standaard instelling, waardoor je simulatie automatisch begint te lopen voor 1000 ns alvorens je de kans krijgt om inwendige signalen toe te voegen aan je simulatievenster. Om dit te vermijden, klik je best eerst met de rechtermuisknop op “Simulation” en vervolgens op “Simulation Settings”. Dit opent een nieuw venster waar je in het linker paneel “Simulation” (onder “Project Settings”) moet kiezen, zodat je vervolgens in het rechter paneel het tabblad “Simulation” kan kiezen en de 1000 ns achter “xsim.simulate.runtime” kan verwijderen. Vanaf dan zal de simulatie niet meer automatisch beginnen lopen van zodra je ze opstart. Ter herinnering kan je best ook eerst nog even de inleidende opgave van het vak 1-Basis digitale elektronica (zie Blackboard) overlopen, zodat je de verschillende knoppen op de interface beter begrijpt. Dit zal je veel tijd besparen om je simulaties te begrijpen en de opgaven op te lossen.

Bekijk zeker ook de demo [“Simuleren met Vivado” op de Mediasite van UA](#).

● Opgave 2

Gebruik de assembler code die bij deze opgave hoort om opnieuw een simulatie te lopen. Ditmaal wordt het timer blok gebruikt. Wijzig het blokschema uit dit document om het timer blok correct toe te voegen aan het ontwerp: Teken de hardware (mux, or/and/not) waarmee er gelezen en geschreven kan worden naar dit blok en vind deze ook terug in de VHDL.

¹ De testbench code in commentaar hoeft je voorlopig nog niet te gebruiken. In opgave 5 kan je die actief maken om je ontwerp automatisch te laten testen. Zorg er ook voor dat je deze code begrijpt.

● Opgave 3

Pas het ontwerp zodanig aan dat er een extra instructie ondersteund wordt, namelijk de instructie BitOr. Deze instructie berekent, gelijkaardig aan de BitAnd instructie, de bitgewijze OF-functie tussen twee operands. Je zal hiervoor de VHDL moeten wijzigen, maar uiteraard ook de ASM code. Verifieer de werking in simulatie.

Als dat gelukt is, doe dan hetzelfde voor de BitOrZ instructie (wijzig de vorige of voeg ze toe). Deze instructie werkt volledig hetzelfde als BitOr, maar doet dit enkel wanneer de Z-vlag gezet is (1 is). Wanneer de Z-vlag niet op 1 staat, mag de instructie niet uitgevoerd worden (de ACC moet ongewijzigd blijven) én mogen de C- en Z-vlaggen niet aangepast worden!

● Opgave 4

Schrijf een assembler programma dat enkel de timer reset, zonder de andere bits van het timer control register te wijzigen. Hiervoor zal je je nieuwe BitOr instructie van opgave 3 nodig hebben.

● Opgave 5

Schrijf een assembler programma dat de waarden op de input poort uitleest en interpreteert als een detector van een vloeistofpeil. De bits zijn verbonden met de uitgangen van vloeistofdetectors, waarbij de meest linkse bit (bit 7) helemaal onderaan het vloeistofreservoir zit en de meest rechtse bit (bit 0) helemaal bovenaan. Dit wil dus zeggen dat bij een binaire waarde 11110000 het reservoir ongeveer halfvol is. Aan de hand van deze sensorwaarden kan je nu verschillende zones definiëren: groen, oranje en rood (zie onderstaande tabel). Je programma moet de sensorwaarden uitlezen en de overeenkomstige zone op de output poort zetten, zoals in de tabel gespecificeerd. Vanaf het ogenblik dat er een "onmogelijke" waarde verschijnt op de sensorwaarden (bv. 11011000) moet het systeem in alarm gaan en er niet meer uit komen tot er een volledige reset is gebeurd. Dit wil namelijk zeggen dat minstens één van de sensoren defect is en dat die eerst hersteld moet worden.

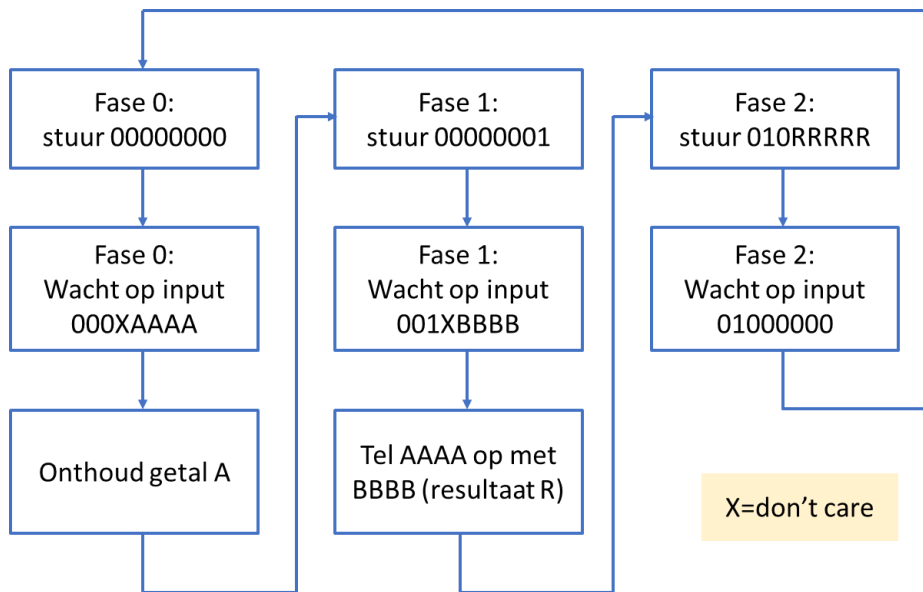
Input port bits								zone	Output port bits				
7	6	5	4	3	2	1	0		7	6	5	4	3-0
0	0	0	0	0	0	0	0	rood	0	0	1	0	0
1	0	0	0	0	0	0	0	rood	0	0	1	0	0
1	1	0	0	0	0	0	0	oranje	0	1	0	0	0
1	1	1	0	0	0	0	0	oranje	0	1	0	0	0
1	1	1	1	0	0	0	0	groen	1	0	0	0	0
1	1	1	1	1	0	0	0	oranje	0	1	0	0	0
1	1	1	1	1	1	0	0	oranje	0	1	0	0	0
1	1	1	1	1	1	1	0	rood	0	0	1	0	0
1	1	1	1	1	1	1	1	rood	0	0	1	0	0
vanaf andere waarde								alarm	0	0	0	1	0

Je zal merken dat je hiervoor veel baat zou hebben met nóg een extra instructie, namelijk een "compare" instructie. Deze is volledig gelijkaardig aan de SUB instructie, in die zin dat ze exact dezelfde aftrekking doet, maar in plaats van het resultaat van de bewerking in de accumulator te zetten, worden enkel de Z- en C-vlaggen aangepast. Hierdoor blijft de waarde van de accumulator ongewijzigd, wat zeer interessant zal blijken voor deze opgave. Je bent volledig vrij om de opgave op te lossen zonder deze nieuwe instructie, maar je zal zien dat je programma aanzienlijk ingekort kan worden mét deze COMP instructie.

== ?

● Opgave 6

Schrijf een assembler programma dat de volgende flowchart implementeert:



Dit programma berekent dus de optelling van getal A en B via de input port en de output port van de SimpleCPU via verschillende fases.

Het “sturen” gebeurt via de OUTPUT ACC instructie op de output port en de input komt van de gebruiker op de input port (via de INPUT ACC instructie). “Wachten” gebeurt in een lus en je mag niet verder gaan in de flowchart vooraleer de betreffende voorwaarde voldaan is. “Onthouden” zal je ergens in het geheugen moeten doen (READ/WRITE MEM).

Let op dat een X duidt op een “don’t care” en dus eender welke waarde kan hebben. Daar moet je dus rekening mee houden. In het “input” gedeelte van fase 0 mag je er dus niet vanuit gaan dat de X gelijk zal zijn aan 0. Een gebruiker kan daar perfect 00010101 sturen, waarbij A dus 5 moet zijn.

Een tip: Gebruik de “ADD IND mm” instructie.

Debug je programma aan de hand van een testbench.